

Lab 4: Working with Amazon Elastic Block Store (EBS)

Objective

The main goal of this lab was to understand how Amazon Elastic Block Store (EBS) works as persistent storage for EC2 instances. I created a new EBS volume, attached it to an existing instance, formatted it, mounted it, wrote some data, took a snapshot for backup, and then restored that snapshot to a new volume to see how data recovery works. **Key AWS services and concepts covered:**

- Amazon Elastic Block Store (EBS)
- Creating and attaching EBS volumes
- Formatting and mounting volumes in Linux
- Updating `/etc/fstab` for persistent mounting
- Creating point-in-time snapshots
- Restoring data from snapshots

Scenario

In this lab there was already an EC2 instance called “Lab” running. I had to create a small additional storage volume (1 GiB), connect it to that instance, set it up so the operating system could use it, save some data, take a backup (snapshot), delete the test file to simulate data loss, and then bring the data back using a new volume created from the snapshot.

Lab Architecture Overview

The diagram below shows the basic setup I worked with: one EC2 instance with its original root volume, plus the new EBS volume I created and attached, and later the restored volume from the snapshot.

EC2 Instance with EBS Volumes

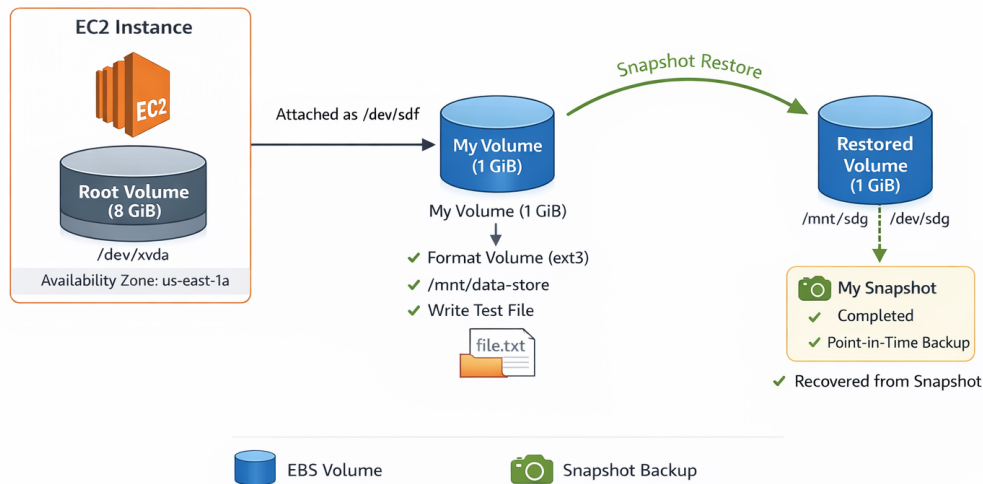


Figure 1: Overview of EC2 instance with root volume and additional EBS volume attached (later restored from snapshot)

This picture helped me understand that EBS volumes are separate from the instance itself and can survive even if something happens to the EC2 server.

Working Methodology

Task 1 & 2: Creating and Attaching the EBS Volume

I went to the EC2 console, checked that the “Lab” instance was in us-east-1a, then created a new 1 GiB gp2 volume in the same availability zone and gave it the name tag “My Volume”. After it became Available, I attached it to the Lab instance using device name /dev/sdf.

Observations:

- The volume quickly changed from Creating to Available.
- After attaching, it showed In-use status.

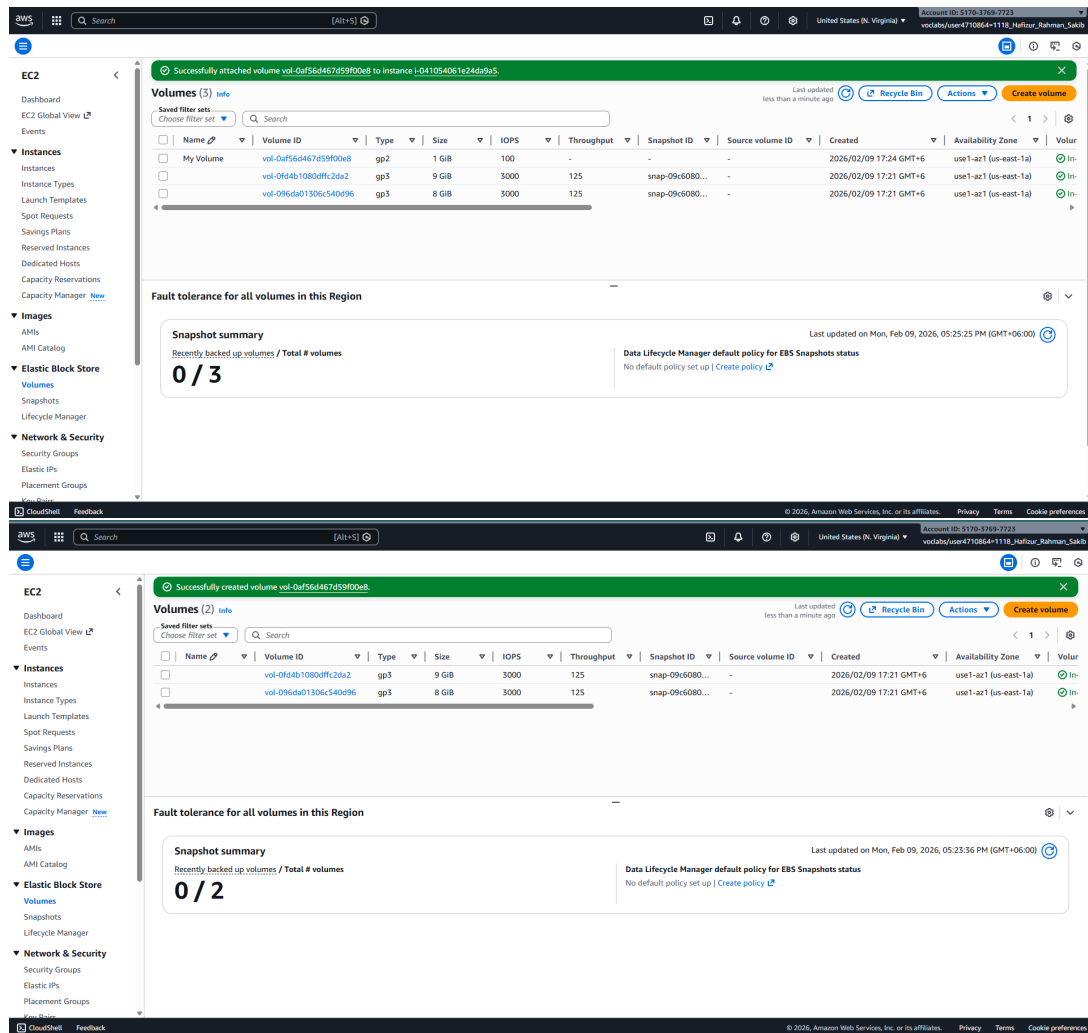


Figure 2: EBS Volumes list showing the new 1 GiB “My Volume” attached to the Lab instance

This screenshot shows both the original 8 GiB root volume and my new volume side by side, which helped me confirm everything was set up correctly.

Task 3 & 4: Connecting and Preparing the Volume

I used EC2 Instance Connect to open a terminal to the Lab instance. First I ran `df -h` and saw only the root volume. Then I created an ext3 file system on `/dev/sdf`, made a mount point `/mnt/data-store`, mounted the volume, and added an entry to `/etc/fstab` so it would mount automatically on reboot. After that, `df -h` showed the new volume correctly mounted. I created a test file with some text inside.

```
newer release of "Amazon Linux" is available.
Version: 2023.10.20260202
Run "/usr/bin/dnf check-release-update" for full release and version update info

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

ec2-user@ip-10-1-11-79 ~$

i-041054061e24da9a5 (Lab)
PublicPis: 13.222.102.63 PrivatePis: 10.1.11.79

CloudShell Feedback
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

AWS
[Alt+S] Ask Amazon Q

mpfs 481M 0 481M 0% /dev/shm
mpfs 153M 448K 192M 1% /run
/dev/xvda1 8.0G 1.6G 6.4G 20% /
mpfs 481M 0 481M 0% /tmp
/dev/xvda128 10M 1.3M 8.7M 13% /boot/efi
mpfs 97M 0 97M 0% /run/user/1000
ec2-user@ip-10-1-11-79 ~$ sudo mkfs -t ext3 /dev/mdf
mkfs: 1.4k:5 (3k=200=2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
filesystem UUID: 6eaa0ff3-2111-4988-b898-778de7ec973d
superblock backups stored on blocks:
32768, 98304, 163840, 229376
Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
ec2-user@ip-10-1-11-79 ~$ sudo mkdir /mnt/data-store
ec2-user@ip-10-1-11-79 ~$ sudo mount /dev/mdf /mnt/data-store
ec2-user@ip-10-1-11-79 ~$ echo /dev/mdf /mnt/data-store ext3 defaults,noatime 1 2 | sudo tee -a /etc/fstab
/dev/mdf /mnt/data-store ext3 defaults,noatime 1 2
ec2-user@ip-10-1-11-79 ~$ cat /etc/fstab
#hash: $'E[200=cat'; command not found
ec2-user@ip-10-1-11-79 ~$ cat /etc/fstab
UUID=298c4de4-f9e5-428a-9b9f-c0c40bf3079 / xfs defaults,noatime 1 1
UUID=8b3-6A0A /boot/efi efat defaults,noatime,uid=0,gid=0,umask=0077,shortname=winnt,x-systemd.automount 0 2
/dev/mdf /mnt/data-store ext3 defaults,noatime 1 2
ec2-user@ip-10-1-11-79 ~$ df -h
Filesystem Size Used Avail Use% Mounted on
devtmpfs 4.0M 0 4.0M 0% /dev
tmpfs 481M 0 481M 0% /dev/shm
tmpfs 153M 448K 192M 1% /run
/dev/xvda1 8.0G 1.6G 6.4G 20% /
mpfs 481M 0 481M 0% /tmp
/dev/xvda128 10M 1.3M 8.7M 13% /boot/efi
mpfs 97M 0 97M 0% /run/user/1000
/dev/xvdf 97M 60K 904M 1% /mnt/data-store
ec2-user@ip-10-1-11-79 ~$
```

Figure 3: Output of `df -h` command after mounting the new volume at `/mnt/data-store`

This picture proves that the volume was properly formatted, mounted, and usable. Seeing `/dev/xvdf` appear with 976M capacity made it clear the steps worked.

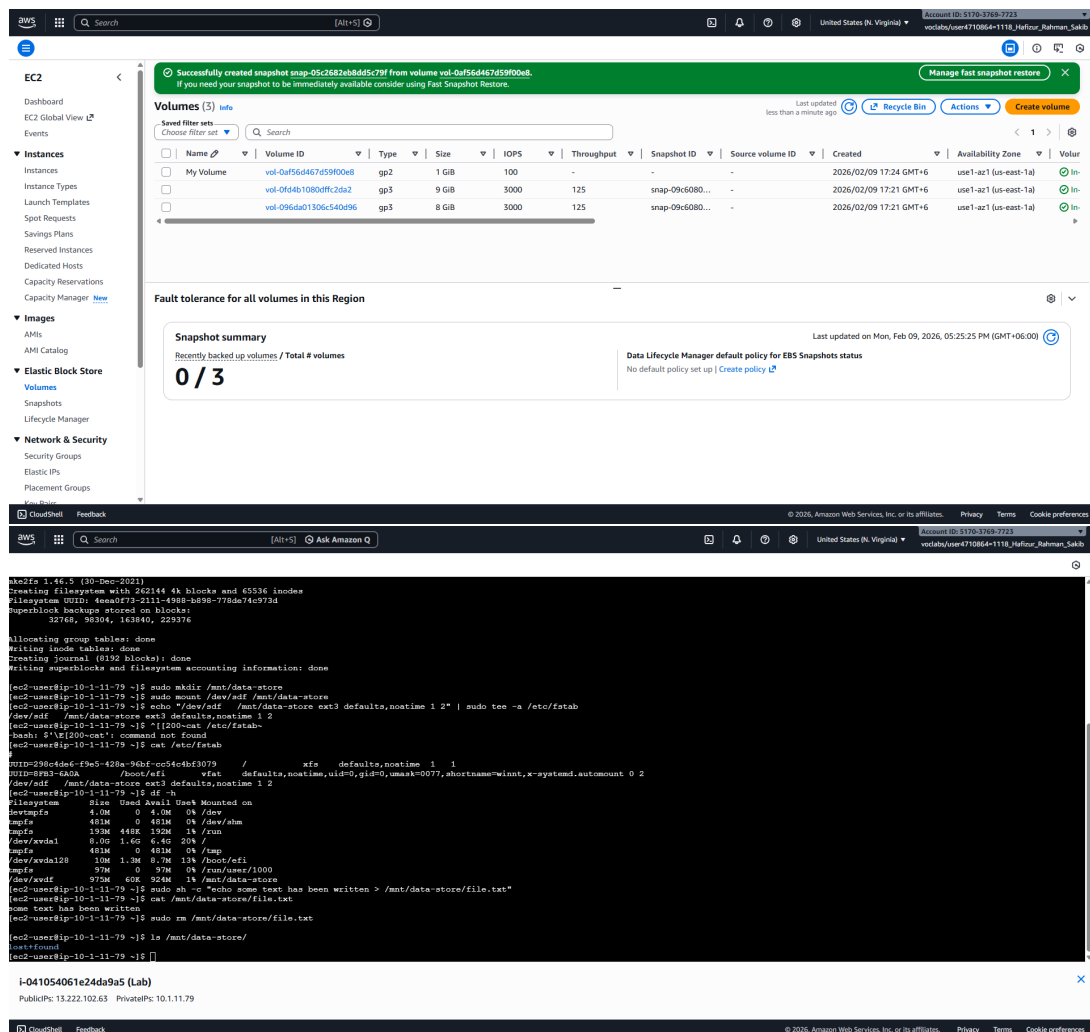


Figure 4: Content of /etc/fstab file showing the new line added for persistent mounting

Adding the line to fstab was important — I learned that without it the volume would not mount after a restart.

Task 5: Creating a Snapshot

I went back to the console, selected My Volume, and created a snapshot named “My Snapshot”. It went from Pending to Completed quickly. Then in the terminal I deleted the test file I had created earlier to simulate losing data.

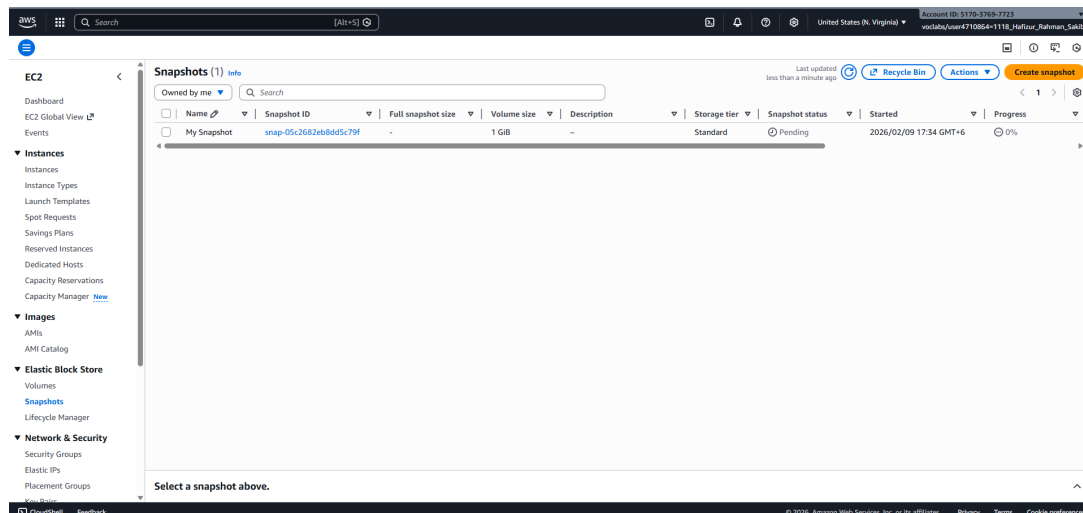


Figure 5: Snapshots section showing “My Snapshot” in Completed state

This screenshot confirms the backup was successful. I found it interesting that snapshots only store the blocks that have data, so empty space does not waste storage.

Task 6: Restoring from Snapshot

I selected the snapshot and created a new volume from it, naming it “Restored Volume” in the same AZ. After it was available, I attached it to the Lab instance as `/dev/sdg`. In the terminal I created a new mount point `/mnt/data-store2`, mounted `/dev/sdg` there, and checked — the `file.txt` was back!

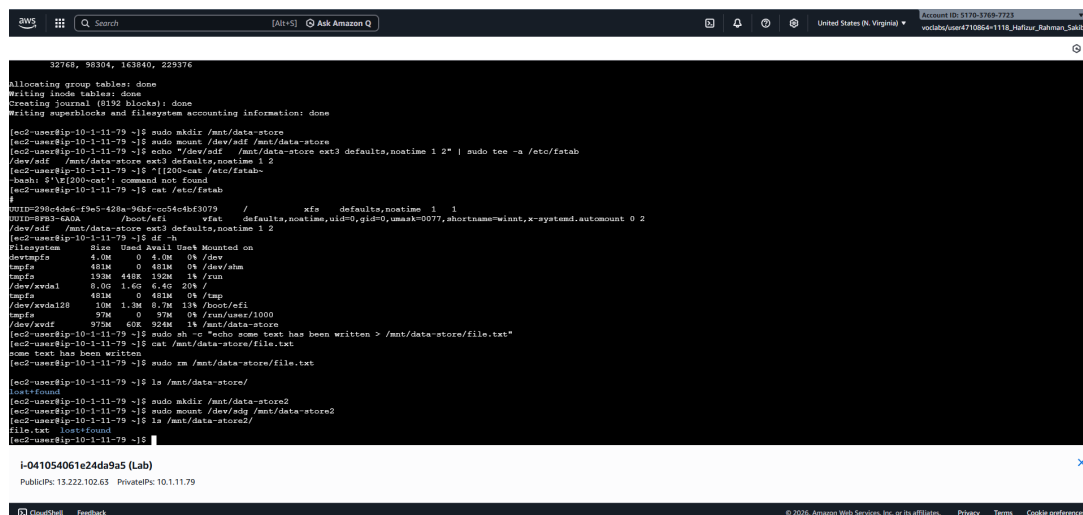


Figure 6: Figure 6: Contents of `/mnt/data-store2` showing `file.txt` restored from the snapshot

This last screenshot was exciting — it proved that the snapshot really worked as a backup and I could recover the data perfectly.

Results Summary

Completed Tasks List as Per Requirements:

- Created and attached 1 GiB EBS volume
- Formatted, mounted, and made persistent via fstab
- Wrote test data and verified
- Created snapshot
- Deleted test file
- Restored snapshot to new volume and recovered the file

Conclusion

This lab helped me understand that EBS volumes are very useful for keeping data safe even if the EC2 instance stops or has problems. I liked seeing how easy it is to attach extra storage, format it, and use it like a normal disk. The snapshot feature was the most interesting part — it gave me confidence that I can protect important files and recover them when needed. Overall this lab made the idea of block storage in the cloud much clearer to me and showed why it is more reliable than instance store.